Popravni test iz predmeta
Algoritama i struktura podataka

Drugi praktični test održan 06.02.2024.

Ime i prezime: _____

Broj indeksa: _____

Broj bodova: _____

Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je *closed book test*, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
    a. Vodite računa da imate sve spremno što vam je neophodno.
    b. Isključite mobilne telefone i odložite ih na katedru.
4. Ukupan broj bodova na testu je 35 bodova.
5. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
6. U pruženom šablonu obavezno upisati broj indeksa na nazbačenom mestu.
7. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP2023_Test2_GrupaA_IT1g2021**
8. Ne postoji minimum bodova koji se mogu osvojiti iz svakog pojedinačnog zadatka
9. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova**

Pre početka testa napišite ime i prezima i broj indeksa.

PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

Zadatak 1 – *Implementacija DFSVisit metode Depth-First-Search algoritma*

Napisati implementaciju DFSVisit metode istoimenog algoritma.

Dodatne napomene za rešavanje zadatka:

- U template-u je zakomentarisana metoda ***private void DFSVisit(LinkedList.Node node, int nodeIndex)*** koja je osnovna metoda za rekurzivno izvršavanje DFS algoritma.
- U *Main* metodi obezbediti način za testiranje i proveru nove funkcionalnosti.

Zadatak 2 – *Zamena ponavljajućih i neponavljajućih karaktera*

Napisati algoritam vremenske kompleksnosti O(n) koji će iz string-a koji se prosleđuje razdvaja ponavljajuće od neponavljajućih karaktera i zamenjuje ih prema sledećem šablonu:

- Ponavljajući karakter = „@“
- Neponavljajući karakter = „#”

Dodatne napomene za rešavanje zadatka:

- Koristiti *Hash* tabelu koja je data u šablonu.
- Implementaciju pisati u okviru zakomentarisane metode ***public static string changeAllDuplicates(string str).***
- Algoritam realizovati tako da **ne postoji** razlika između malih i velikih karaktera.
- Kako biste string konvertovali u niz karaktera sa kojim se može vršiti manipulacija, koristiti metodu ***ToCharArray()*** po principu: ***str.ToCharArray().***
- Karakteri se ne moraju eksplicitno konvertovati u celobrojni tip prilikom ubacivanja u *Hash* tabelu, ali prilikom kasnijeg ispisivanja koristiti eksplicitno kastovanje ***(char).***
- Redosled karaktera prilikom ispisivanja nije bitan.

Primer:

Input 1: *“Algoritam”*

Output 1: *“@######@#“*

(Samo karakter „A“ se pojavljuje više puta)

Input 2: *“Strukture podataka”*

Output 2: *“#@@@@@@@# ###@@@@“*

(Karakteri „a“, „r“, „u“, „k“ i „t“ se pojavljuju više puta)

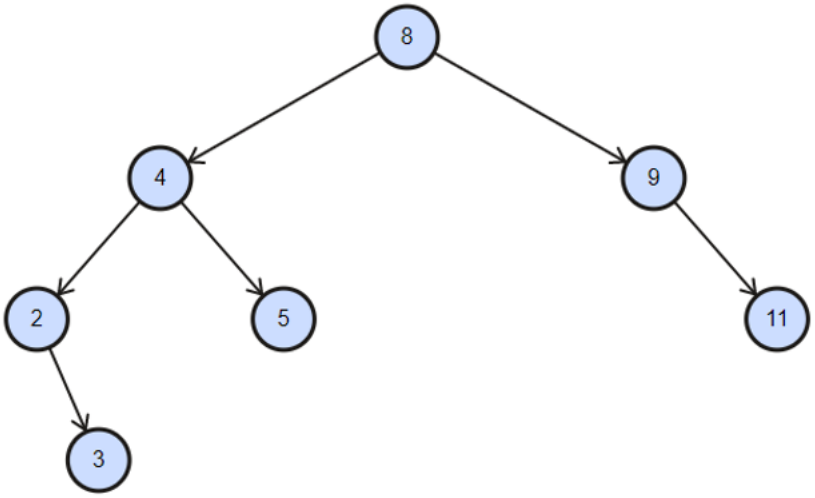
Zadatak 3 – Pronalaženje razlike čvorova na parnim i neparnim nivoima stabla

U okviru standardne strukture klase ***BinarySearchTree*** napisati implementaciju metode koja će vratiti vrednost koja se dobija oduzimanjem vrednosti koje se nalaze u čvorovima na neparnim nivoima od vrednosti koje se nalaze u čvorovima na parnim novima stabla.

Dodatne napomene za rešavanje zadatka:

- Koren stabla se nalazi na nultom nivou.
- U okviru klase ***BinarySearchTree*** kreirati javnu metodu sa potpisom ***public int findDiffEvensOdds()***.
- Iz prethodno kreirane javne metode treba da se poziva privatna metoda sa potpisom ***private int findDiffEvensOdds(Node curr, bool levelIsEven)***
- Privatna metoda treba da sadrži implementaciju algoritma kojim će se omogućiti računanje razlike vrednosti čvorova na parnim i neparnim nivoima stabla
- Algoritam **OBAVEZNO** pisati u okviru klase ***BinarySearchTree***

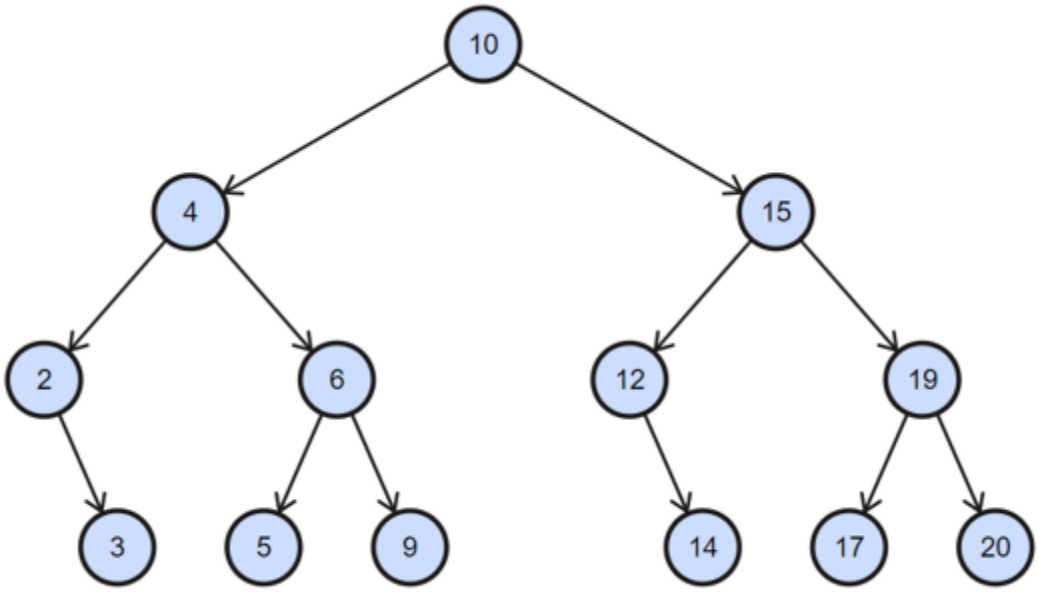
Primeri:



Slika 1 - Primer BST-a

Input 1: *BST sa slike 1*

Output 1: *10*



Slika 2 - Primer BST-a

Input 2: *BST sa slike 2*

Output 2: *-38*